Software Testing Preparation

Generated: 2026-11-01T19:02:44.858Z • Typeset for Academic Review

POWERED BY
College Study
Your Study Hub

USER

Listen, yeh mid sem 2 , software testing ka question paper hai last year ka , to isbaar kon Kon se questions aa sakte hai yeh bhi bata dena in detail answer ke sath in totally in english OK and pyqs ke answers bhi OK. Unit 3 & 4 aa rahi hai exam me ...

ASSISTANT

SOFTWARE TESTING – MID-SEM 2 COMPLETE PREPARATION

(Units 3 & 4) (Previous Year Questions + Expected Questions)

I have gone through both images carefully. We will prepare this from an examination point of view, keeping 5-mark answers, important differences, theoretical questions and practical case studies in mind.

One important thing: I cannot guarantee the exact questions that will appear, but based on your syllabus and last year's paper, I can identify the most important and probable questions.

1. Understanding the exam pattern

Your previous-year paper contains three major question styles:

1. Write short notes: Definitions, features, requirements and explanations.
2. Differentiate between: Two concepts with at least 4–5 comparison points.
3. Case-study based question: Apply software testing concepts to a practical situation.

Therefore, prepare all three formats.

2. Most important topics – Unit 3 and Unit 4

UNIT 3 – Integration, System and Acceptance Testing

(High priority)

1. Top-down and Bottom-up Integration Testing
2. Bi-directional Integration Testing
3. System Integration Testing
4. Scenario Testing
5. Defect Bash
6. Functional vs Non-functional Testing
7. Design vs Architecture Verification
8. Deployment Testing
9. Beta Testing
10. Scalability, Reliability and Stress Testing
11. Acceptance Testing and Acceptance Criteria
12. Test Case Selection and Execution

UNIT 4 – Test Selection & Minimization for Regression Testing

(High priority)

1. Regression Testing and its Process
2. Smoke Testing and Sanity Testing
3. Selection of Regression Tests
4. Execution Trace
5. Dynamic Slicing
6. Test Minimization

7. Tools for Regression Testing
8. Ad hoc Testing
9. Pair Testing
10. Exploratory Testing
11. Iterative Testing
12. Defect Seeding

UNIT 3 – Integration, System and Acceptance Testing

Q1. What is Integration Testing? Explain its different approaches.

Answer:

Integration Testing is a level of software testing in which two or more individually tested modules are combined and tested together to identify defects in their interactions.

The main objective is to verify whether different modules communicate and exchange data correctly.

For example, in an online shopping application:

- Login module
- Product module
- Payment module
- Order module

These modules may work independently, but integration testing checks whether they work correctly when connected.

Types of Integration Testing

1. Top-Down Integration Testing

In this approach, testing starts from the top-level module and gradually moves towards lower-level modules.

- Higher-level modules are tested first.
- Lower-level modules are replaced by temporary programs called stubs.
- Stubs simulate the behaviour of lower-level modules.
- Gradually, actual modules replace the stubs.

Example: Testing the Login module first and then integrating the User Profile and Database modules.



Diagram: top down integration testing software engineering diagram stubs hierarchy

Advantages:

- High-level logic is tested early.
- Early detection of architectural defects.
- No need for drivers.

Disadvantages:

- Requires stubs.
- Lower-level modules are tested late.
- Detailed functionality may remain untested initially.

2. Bottom-Up Integration Testing

In this approach, testing starts from the lowest-level modules and gradually moves towards higher-level modules.

- Lower-level modules are tested first.
- Temporary programs called drivers are used.
- Drivers simulate the behaviour of higher-level modules.
- Modules are gradually combined until the complete system is formed.

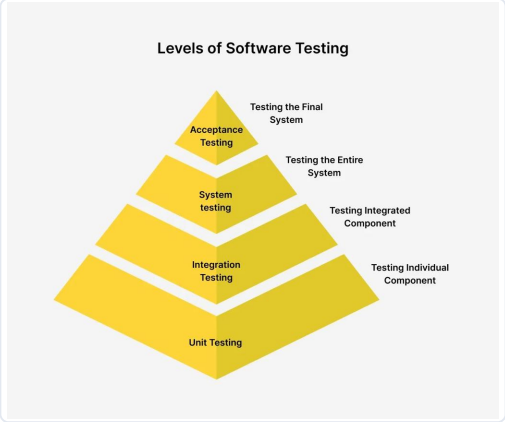


Diagram: bottom up integration testing diagram drivers software engineering

Advantages:

- Low-level functionality is tested early.
- No need for stubs.
- Easier identification of defects in lower modules.

Disadvantages:

- High-level logic is tested late.
- Requires drivers.
- System functionality is not visible in the early stages.

3. Bi-directional Integration Testing

Bi-directional integration testing combines both top-down and bottom-up approaches.

- Testing starts from both higher-level and lower-level modules.
- Both approaches proceed towards the middle layer.
- The middle modules are integrated and tested.
- It helps detect defects at different levels.

Advantages:

- Faster testing of different layers.
- Combines benefits of both approaches.
- Useful for large and complex applications.

Conclusion: Integration testing ensures that individually developed modules work correctly when combined.

Q2. What is System Integration Testing (SIT)?

Answer:

System Integration Testing is a testing technique used to verify the interaction and communication between different systems, applications or external services.

It checks whether the complete system works correctly when connected with other systems.

Example:

Consider an online banking application connected with:

- Payment Gateway
- Banking Database
- SMS Notification Service
- Email Service

System Integration Testing checks whether these external systems communicate correctly.

Objectives:

1. Verify data exchange between different systems.
2. Check communication interfaces.
3. Verify API interactions.
4. Identify integration defects.
5. Ensure compatibility between connected systems.
6. Verify error handling during communication failures.

Advantages:

- Detects interface-related defects.
- Improves system compatibility.
- Ensures correct data exchange.
- Increases overall system reliability.

Q3. Explain Scenario Testing with an example.

Answer:

Scenario Testing is a software testing technique in which complete real-world user scenarios are tested to verify whether the application performs its expected functions.

Instead of testing individual functions separately, it tests a sequence of related activities.

Example: Online Shopping Application

A complete test scenario could be:

1. User opens the shopping application.
2. User logs into the account.
3. User searches for a product.
4. User adds the product to the cart.
5. User enters the delivery address.
6. User completes the payment.
7. User receives order confirmation.

If any step fails, the complete scenario may fail.

Characteristics:

- Based on real-world situations.
- Focuses on end-to-end functionality.
- Tests multiple connected modules.
- Helps identify defects in user workflows.

Advantages:

- Tests actual user behaviour.
- Identifies integration issues.
- Improves user experience.
- Covers complete business processes.

Q4. What is Defect Bash?

Answer:

Defect Bash is a testing activity in which multiple team members, such as developers, testers and sometimes business users, test an application together within a limited period to identify as many defects as possible.

It is generally performed before a major release.

Process:

1. Select the application or module to be tested.
2. Define the testing scope.
3. Provide access to the testing environment.
4. Assign different areas to team members.
5. Test the application using different inputs and scenarios.
6. Record and report all identified defects.
7. Prioritize defects according to severity.
8. Resolve and retest the defects.

Advantages:

- Identifies hidden defects.
- Encourages teamwork.
- Provides different testing perspectives.
- Helps improve software quality before release.

Example: Before releasing a college management system, developers and testers simultaneously test student registration, fee payment, attendance and result modules.

Q5. Differentiate between Functional and Non-functional Testing.

(Very important – PYQ)

Answer:

Functional Testing	Non-functional Testing
Tests what the system does.	Tests how well the system works.
Checks functional requirements.	Checks quality attributes.
Focuses on features and operations.	Focuses on performance, security and usability.
Verifies expected outputs.	Verifies system behaviour under different conditions.
Usually based on business requirements.	Usually based on quality requirements.
Example: Login functionality.	Example: Login response time.

Examples of Functional Testing:

- Login testing
- Registration testing
- Payment testing
- Search functionality testing

Examples of Non-functional Testing:

- Performance testing
- Load testing
- Stress testing
- Security testing
- Usability testing
- Reliability testing

Conclusion: Functional testing verifies whether the software performs the required functions, whereas non-functional testing verifies the quality and performance of those functions.

Q6. Differentiate between Design Verification and Architecture Verification.

Answer:

Design Verification	Architecture Verification
Checks whether the detailed software design is correct.	Checks whether the overall system architecture is correct.
Focuses on individual components and their implementation design.	Focuses on the overall structure of the system.
Examines classes, methods, interfaces and algorithms.	Examines components, layers and their relationships.
Checks whether design specifications are followed.	Checks whether architectural requirements are satisfied.
Usually performed at a detailed design level.	Usually performed at a high-level system design level.

Example:

- Design verification: Checking whether the login function correctly validates username and password.
- Architecture verification: Checking whether the application correctly follows a three-tier architecture consisting of presentation, business logic and database layers.

Q7. Explain Deployment Testing.

Answer:

Deployment Testing is the process of verifying whether software can be correctly installed, configured and executed in the target environment.

It ensures that the application works properly after deployment.

Important activities:

1. Verify software installation.
2. Check system configuration.
3. Verify database connectivity.
4. Check compatibility with the operating system.
5. Verify environment variables and dependencies.
6. Check application startup and shutdown.
7. Verify backup and recovery procedures.
8. Test application functionality after deployment.

Example: After deploying a college ERP system on a college server, testers verify that students can log in, access attendance and download results.

Q8. What is Beta Testing? Explain its advantages.

Answer:

Beta Testing is a type of software testing in which a nearly complete version of the software is released to a limited group of real users outside the development organization.

The objective is to collect real-world feedback and identify defects before the final release.

Process:

1. Develop a nearly complete software product.
2. Select a group of beta users.
3. Distribute the beta version.
4. Allow users to use the application in real environments.
5. Collect feedback and bug reports.
6. Analyse and fix reported defects.
7. Release the final version.

Advantages:

- Provides real-user feedback.

- Identifies defects missed during internal testing.
- Checks usability and compatibility.
- Improves product quality.
- Reduces the risk of failure after release.

Example: A mobile application is provided to 500 selected users before its official launch to collect feedback.

Q9. Explain Scalability Testing, Reliability Testing and Stress Testing.

Answer:

1. Scalability Testing

Scalability Testing checks whether a system can handle an increasing workload by adding resources such as CPU, memory or servers.

Example: Checking whether a website can handle an increase from 1,000 to 50,000 users.

Objectives:

- Measure system performance as workload increases.
- Identify resource limitations.
- Check whether the system can grow efficiently.

2. Reliability Testing

Reliability Testing checks whether software can perform its required functions continuously and without failure for a specified period under specified conditions.

Example: Checking whether an online banking system operates continuously for 72 hours without unexpected failure.

Objectives:

- Measure failure frequency.
- Identify recurring failures.
- Check system stability.
- Improve system availability.

3. Stress Testing

Stress Testing checks how a system behaves when subjected to workloads beyond its normal operating capacity.

Example: Testing a website with 1 lakh simultaneous users when it is designed for 20,000 users.

Objectives:

- Identify the breaking point.
- Check system stability under extreme conditions.
- Verify error handling.
- Observe recovery after excessive workload.

Difference between the three:

Scalability Testing	Reliability Testing	Stress Testing
Checks ability to handle increasing workload.	Checks consistent operation over time.	Checks behaviour under extreme workload.
Focuses on growth.	Focuses on stability.	Focuses on system limits.
Resources may be increased.	Normal operating conditions are generally maintained.	Normal limits are deliberately exceeded.

Q10. What is Acceptance Testing? Explain its types.

Answer:

Acceptance Testing is the final level of software testing performed to determine whether the software satisfies the specified business requirements and is ready for delivery or use.

It is generally performed by customers, end users or their representatives.

Objectives:

1. Verify business requirements.
2. Check whether the software meets user expectations.
3. Identify major issues before release.
4. Ensure readiness for production.
5. Obtain customer approval.

Types of Acceptance Testing

1. User Acceptance Testing (UAT): Performed by actual users to verify whether the application meets their business needs.

2. Alpha Testing: Conducted at the developer's site by internal testers or selected users before beta testing.

3. Beta Testing: Conducted by external users in real-world environments.

4. Operational Acceptance Testing: Checks operational readiness, including backup, recovery, security, maintenance and deployment.

5. Contractual Acceptance Testing: Verifies whether the software satisfies the acceptance conditions mentioned in the contract.

6. Regulatory Acceptance Testing: Checks compliance with applicable laws, regulations and standards.

Q11. What are Acceptance Criteria? Explain with an example.

Answer:

Acceptance Criteria are predefined conditions that a software product or feature must satisfy to be accepted by the customer or end user.

They define the minimum requirements that must be fulfilled before a feature is considered complete.

Example: Login System

Acceptance criteria:

1. The user must be able to log in using valid credentials.
2. Invalid credentials must generate an appropriate error message.
3. The password must be stored securely.
4. The system must provide a password recovery option.
5. The login page must work on supported devices.
6. The user must be redirected to the dashboard after successful login.

Importance:

- Defines clear expectations.
- Reduces misunderstandings.
- Helps testers prepare test cases.
- Provides a basis for acceptance decisions.
- Improves communication between developers and customers.

Q12. Explain Test Case Selection and Execution.

Answer:

Test Case Selection is the process of choosing appropriate test cases from a test suite to verify specific software requirements or functionalities.

Test Execution is the process of running selected test cases and comparing actual results with expected results.

Steps involved:

1. Analyse software requirements.
2. Identify important functionalities.
3. Prepare test cases.
4. Prioritize test cases according to risk and importance.

5. Select relevant test cases.
6. Prepare the testing environment.
7. Execute selected test cases.
8. Compare actual and expected results.
9. Record pass or fail status.
10. Report defects and perform retesting.

Example:

For an online payment system, test cases may include:

- Successful payment.
- Payment with insufficient balance.
- Invalid card details.
- Network failure during payment.
- Duplicate payment request.

All these test cases verify different situations in the payment process.

UNIT 4 – Test Selection & Minimization for Regression Testing

(Very important unit)

Q1. What is Regression Testing? Explain its process.

Answer:

Regression Testing is a type of software testing performed after making changes to an existing software application to ensure that previously working functionalities are still working correctly.

Whenever developers modify existing code, add new features or fix bugs, there is a possibility that these changes may affect other parts of the application.

Regression testing helps identify such unexpected problems.

Example:

Suppose an online shopping application has the following features:

- Login
- Product search
- Add to cart
- Payment
- Order tracking

Now developers add a new discount feature.

Regression testing checks whether the existing login, search, cart and payment functionalities are still working correctly after the new feature is added.

Regression Testing Process



Diagram: software regression testing process flowchart code change impact analysis test selection execution defect reporting

Step 1: Analyse Changes

Identify the modifications made to the software, such as new features, bug fixes or code changes.

Step 2: Identify Impacted Areas

Determine which existing modules or functionalities may be affected by these changes.

Step 3: Select Regression Test Cases

Choose relevant test cases from the existing test suite based on the affected areas and risk.

Step 4: Prioritize Test Cases

Give higher priority to critical functionalities, frequently used features and high-risk modules.

Step 5: Execute Test Cases

Run the selected test cases to verify that existing functionalities continue to work.

Step 6: Compare Results

Compare actual results with expected results to identify failures.

Step 7: Report Defects

Report newly discovered defects to developers for correction.

Step 8: Retesting

After developers fix the defects, execute the relevant test cases again.

Advantages of Regression Testing:

1. Maintains software stability.
2. Detects unexpected side effects.
3. Ensures existing features remain functional.
4. Improves software quality.
5. Reduces the risk of failure after updates.

Q2. Differentiate between Smoke Testing and Sanity Testing.

Answer:

Smoke Testing: Smoke Testing is preliminary testing performed on a new software build to check whether its major functionalities are working properly and whether the build is stable enough for further testing.

Sanity Testing: Sanity Testing is a focused type of testing performed after minor changes or bug fixes to verify that the specific modified functionalities are working correctly.

College Study
Your Study Hub

Smoke Testing	Sanity Testing
Checks major functionalities of the application.	Checks specific modified functionalities.
Performed on a new software build.	Performed after minor changes or bug fixes.
Has broad but shallow coverage.	Has narrow but relatively deep coverage.
Determines whether the build is stable for further testing.	Determines whether the particular changes work correctly.
Usually performed before detailed testing.	Usually performed after receiving a modified build.
Example: Checking login, payment and registration.	Example: Checking a fixed password-reset feature.

Remember: Smoke testing checks whether the build is testable, while sanity testing checks whether a particular change works correctly.

Q3. Explain Selection of Regression Tests.

Answer:

Regression Test Selection is the process of selecting appropriate test cases from an existing test suite to verify that modifications in software have not affected previously working functionalities.

Running every test case after every modification may consume considerable time and resources. Therefore, testers select relevant test cases.

Techniques for Regression Test Selection

1. Retest All:

In this technique, all existing test cases are executed again after every modification.

Advantages:

- Provides extensive test coverage.
- Reduces the possibility of missing affected functionalities.

Disadvantages:

- Time-consuming.
- Expensive for large applications.

2. Selective Regression Testing:

Only test cases related to modified or affected modules are selected.

Advantages:

- Saves execution time.
- Reduces testing cost.
- Suitable for large systems.

3. Priority-Based Selection:

Test cases are selected according to their importance, risk and business impact.

For example, payment and login functionalities receive higher priority than less frequently used features.

4. Risk-Based Selection:

Test cases related to high-risk modules are selected first because failures in these modules may cause serious problems.

Conclusion: Regression test selection helps achieve effective testing while reducing unnecessary execution.

Q4. What is an Execution Trace? Explain its importance.

Answer:

An Execution Trace is a record of the sequence of statements, functions, methods or operations executed by a program during a particular test run.

It provides information about the execution path followed by the software.

Example:

Consider the following program:

```
int calculate(int a, int b) {
    int sum = a + b;
    int result = sum * 2;
    return result;
}
```

For input a = 5 and b = 3, the execution trace is:

1. Function calculate() is called.
2. sum = 5 + 3 is executed.
3. result = 8 * 2 is executed.
4. The function returns 16.

Importance of Execution Trace:

1. Helps understand program execution.
2. Identifies which statements were executed.
3. Helps locate defects.
4. Supports regression test selection.
5. Helps analyse program behaviour.
6. Provides information about code coverage.

Q5. What is Dynamic Slicing? Explain with an example.

Answer:

Dynamic Slicing is a program analysis technique used to identify the statements that affect the value of a particular variable at a specific point during a particular program execution.

It considers the actual execution of a program with specific input values.

In simple words, dynamic slicing identifies which executed statements are responsible for producing a particular output.

Example:

```
int main() {
    int a = 10;
    int b = 20;
    int c = a + b;
    int d = 50;
    int e = c * 2;

    cout << e;
}
```

Suppose we want to find the statements that affect the output variable e.

The dynamic slice includes:

- int a = 10;
- int b = 20;
- int c = a + b;
- int e = c * 2;
- cout << e;

The statement int d = 50; is not included because it does not affect the value of e.

Advantages:

1. Helps locate defects.
2. Reduces debugging effort.
3. Identifies relevant statements.
4. Helps understand program behaviour.

5. Supports effective regression testing.

Static Slicing vs Dynamic Slicing:

Static Slicing	Dynamic Slicing
Considers all possible execution paths.	Considers a particular execution.
Does not require a specific input.	Requires specific input values.
May include statements not executed.	Includes statements relevant to actual execution.
Generally broader.	Generally more specific.

Q6. What is Test Minimization? Explain its importance and techniques.

Answer:

Test Minimization is the process of reducing the number of test cases in a test suite while maintaining the required testing coverage.

Large software applications may contain thousands of test cases. Executing all test cases repeatedly requires significant time and resources.

Test minimization helps remove redundant test cases without unnecessarily reducing testing effectiveness.

Techniques of Test Minimization

1. Elimination of Duplicate Test Cases

Remove test cases that verify the same functionality under identical conditions.

2. Coverage-Based Minimization

Select a smaller set of test cases that provides the required statement, branch or requirement coverage.

3. Prioritization-Based Minimization

Select test cases that cover critical and high-risk functionalities.

4. Redundancy Removal

Identify and remove test cases that provide no additional testing value.

Example:

Suppose a test suite contains 100 test cases, but many test cases cover the same functionality.

After analysing coverage and removing redundant cases, testers may select 60 test cases that still provide the required coverage.

Advantages:

- Reduces testing time.
- Reduces execution cost.
- Improves testing efficiency.
- Avoids unnecessary duplication.
- Makes regression testing faster.

Important: Test minimization should not remove important test cases that are necessary for detecting critical defects.

Q7. Explain Tools used for Regression Testing.

Answer:

Regression Testing tools help automate the execution of previously prepared test cases whenever software modifications are made.

They reduce manual effort and help identify defects quickly.

Important Tools



Diagram: Selenium software testing logo

1.

An open-source tool used to automate web browsers and test web application functionalities.
Example: Automated login and registration testing.

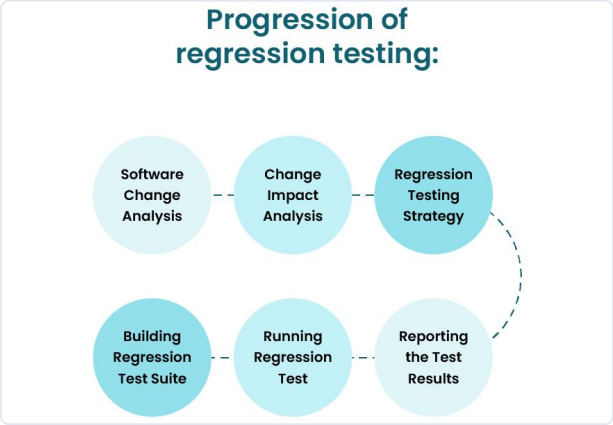


Diagram: JUnit testing framework logo

2.

A Java testing framework used to write and execute automated unit tests.
Example: Testing Java methods after code changes.

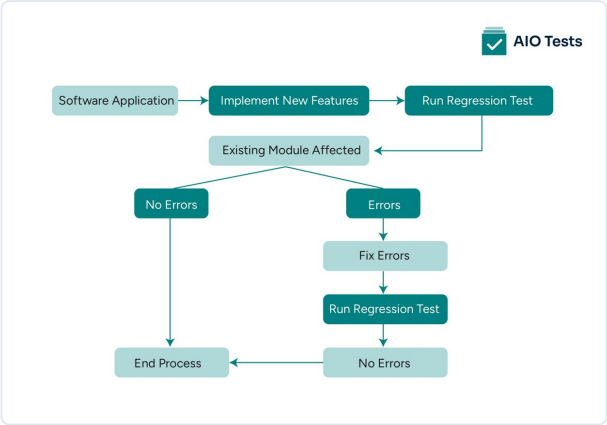


Diagram: TestNG logo

3.

A Java testing framework that supports test grouping, parallel execution and test configuration.

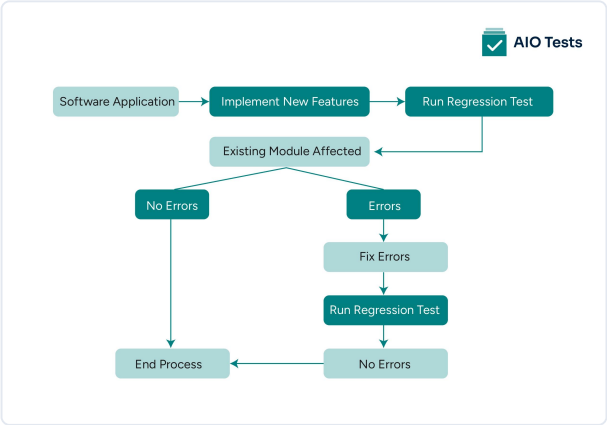


Diagram: Apache JMeter logo

4.

Used to test application performance, load and stress under different conditions.

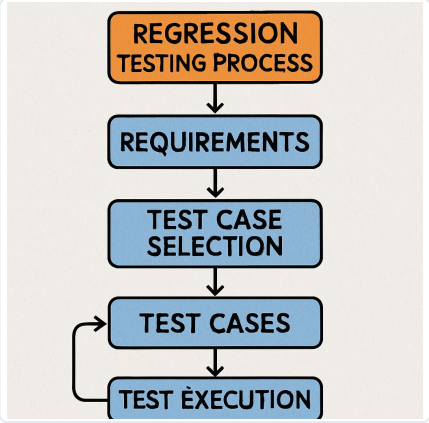


Diagram: Katalon Studio logo

5.

Provides automated testing capabilities for web, mobile, desktop and API applications.

Benefits of Automation Tools:

- Faster test execution.
- Reusable test scripts.
- Reduced manual effort.
- Consistent test execution.
- Easier regression testing.

Q8. Explain Ad hoc Testing.

Answer:

Ad hoc Testing is an informal software testing technique performed without predefined test cases, formal documentation or a structured testing procedure.

Testers use their experience, creativity and intuition to identify defects.

Characteristics:

1. No predefined test cases.
2. No formal documentation is necessary.
3. Based on tester experience.
4. Focuses on finding unexpected defects.
5. Usually performed for a limited period.

Example: A tester randomly enters special characters, extremely long text or unexpected values into a registration form to identify errors.

Advantages:

- Quick defect detection.
- Requires less preparation.
- Helps identify unexpected problems.

Disadvantages:

- Testing is unstructured.
- Difficult to reproduce defects.
- Test coverage is not guaranteed.
- Results may depend on tester experience.

Q9. Explain Pair Testing.

Answer:

Pair Testing is a software testing technique in which two people work together on the same testing activity to identify defects and improve testing effectiveness.

Usually, one person performs the testing while the other observes, analyses and suggests additional test scenarios.

Roles:

1. Tester/Driver: Operates the application and executes test scenarios.
2. Observer/Navigator: Observes the testing process, identifies possible defects and suggests additional test cases.

Both participants can exchange their roles during testing.

Example: Two testers work together to test an online payment system. One performs payment transactions while the other checks transaction details, error messages and possible security issues.

Advantages:

- Better defect identification.
- Knowledge sharing.
- Different testing perspectives.

- Improved communication.
- Reduced chances of missing important scenarios.

Q10. Explain Exploratory Testing.

Answer:

Exploratory Testing is a software testing approach in which testers simultaneously learn about the application, design test cases and execute them.

Unlike traditional testing, where test cases are prepared in advance, exploratory testing allows testers to explore the application and adapt their testing based on observations.

Characteristics:

1. Learning and testing happen simultaneously.
2. Does not depend entirely on predefined test cases.
3. Uses tester experience and creativity.
4. Helps discover unexpected defects.
5. Test scenarios can change during execution.

Example: A tester explores a newly developed mobile application by trying different navigation options, entering unexpected inputs and checking how the application responds.

Advantages:

- Finds hidden defects.
- Provides flexibility.
- Improves understanding of the application.
- Useful when documentation is limited.
- Helps identify usability issues.

Q11. Explain Iterative Testing.

Answer:

Iterative Testing is a testing approach in which software is tested repeatedly during different development iterations.

In every iteration, a particular set of features is developed, tested, evaluated and improved.

Process:

1. Identify the requirements for an iteration.
2. Develop the required functionality.
3. Prepare test cases.
4. Execute testing.
5. Identify and fix defects.
6. Retest the modified functionality.
7. Move to the next iteration.

Example:

For a college management system:

- Iteration 1: Test student registration.
- Iteration 2: Test attendance management.
- Iteration 3: Test fee payment.
- Iteration 4: Test result management.

Each iteration is tested before moving towards further development.

Advantages:

- Early defect detection.
- Continuous improvement.
- Easier management of changes.
- Better software quality.

- Reduced risk of major failures.

Q12. What is Defect Seeding? Explain its purpose.

Answer:

Defect Seeding is a software testing technique in which known artificial defects are deliberately introduced into a program to evaluate the effectiveness of the testing process.

The purpose is to determine how many defects the testing team can detect.

Example:

Suppose developers deliberately introduce 20 artificial defects into a software application.

During testing:

- 15 seeded defects are detected.
- 5 seeded defects remain undetected.

The defect detection effectiveness for seeded defects is:

$$
\text{Detection Rate}=\frac{15}{20}\times100=75\%
$$

Objectives:

1. Evaluate the effectiveness of testers.
2. Measure defect detection capability.
3. Identify weaknesses in testing procedures.
4. Improve testing strategies.
5. Estimate the possibility of undiscovered defects.

Advantages:

- Measures testing effectiveness.
- Helps improve test coverage.
- Evaluates testing techniques.

Limitations:

- Artificial defects may not represent real defects.
- Introduced defects may affect system behaviour.
- Results depend on the quality of seeded defects.

Q13. Explain the Regression Testing process with a suitable diagram.

This question is similar to Q1, but it may be asked as a flowchart-based question.



Diagram: regression testing lifecycle flowchart software testing process

Write the following sequence in your examination:

1. Software modification.
2. Change impact analysis.
3. Regression test selection.
4. Test case prioritization.
5. Test execution.
6. Result comparison.
7. Defect reporting.
8. Retesting and final verification.

---

3. PREVIOUS YEAR QUESTION PAPER – COMPLETE ANSWERS

Now let's solve the questions from your second image, one by one.

Important: A few PYQ topics belong to Unit 5, not your current Unit 3 and Unit 4 syllabus. I am still including their answers so you have the complete previous-year paper solved. Give them lower priority if your teacher has confirmed that only Units 3 and 4 are included.

PYQ – Question 1: Write short notes

(a) Generic Requirements for Test Tool Framework

Answer:

A Test Tool Framework is a structured environment that provides common facilities, libraries, components and guidelines for developing and executing automated software tests.

Generic requirements are the basic features that a testing framework should provide to support effective and reliable testing.

Important Requirements:

1. Test Case Management: It should allow testers to create, modify, organize and maintain test cases.
2. Test Execution: It should support automatic execution of test cases.
3. Test Data Management: It should allow the management of different input data and test datasets.
4. Reporting: It should generate detailed test execution reports.
5. Error Handling: It should identify and record errors during test execution.
6. Reusability: Common functions and test scripts should be reusable.
7. Integration: It should integrate with other development and testing tools.
8. Maintainability: The framework should be easy to modify and maintain.
9. Scalability: It should support an increasing number of test cases.
10. Platform Independence: It should support different operating systems and environments where required.

Conclusion: A good test tool framework reduces manual effort, improves testing efficiency and provides consistent testing results.

(b) Testing Structured Systems and Testing Object-Oriented Systems

Answer:

1. Testing Structured Systems

Structured systems are software systems developed using procedural programming concepts, where a program is divided into functions and modules.

Testing focuses on verifying individual functions, data flow, control flow and interactions between modules.

Important Testing Approaches:

- Unit testing of individual functions.
- Integration testing of connected modules.
- Control-flow testing.
- Data-flow testing.
- Statement and branch coverage.
- Functional testing.

College Study
Your Study Hub

Example: Testing a banking application developed using procedural C programming by checking functi withdrawal and balance enquiry.

2. Testing Object-Oriented Systems

Object-oriented systems are developed using concepts such as classes, objects, inheritance, polymorphism and encapsulation.

Testing focuses on verifying classes, objects, methods, relationships and interactions between objects.

Important Testing Approaches:

- Class testing.
- Method testing.
- Inheritance testing.
- Polymorphism testing.
- State-based testing.
- Object interaction testing.
- Integration testing between classes.

Example: Testing a banking application developed using Java classes such as Account, Customer and Transaction.

Differences:

Structured System Testing	Object-Oriented System Testing
Based on functions and procedures.	Based on classes and objects.
Focuses on functions and modules.	Focuses on classes, methods and objects.
Data and functions are generally separate.	Data and methods are encapsulated together.
Tests control flow and data flow.	Tests object states and interactions.
Inheritance and polymorphism are not central concepts.	Inheritance and polymorphism require special testing.
Example: C-based applications.	Example: Java-based applications.

PYQ – Question 2: Differentiate between the following

(a) Functional Requirements vs Non-functional Requirements

(Unit 3 – Must prepare)

Functional Requirements	Non-functional Requirements
Define what the system should do.	Define how well the system should perform.
Describe system functionalities.	Describe system quality attributes.
Focus on operations and features.	Focus on performance, security and usability.
Usually expressed as business functions.	Usually expressed as measurable quality conditions.
Example: User can log in.	Example: Login must complete within 2 seconds.

(b) Design vs Architecture

(Unit 3 – Must prepare)

Software Design	Software Architecture
Describes how individual components will be implemented.	Describes the overall structure of the system.
Focuses on detailed implementation.	Focuses on high-level organization.
Includes classes, methods and algorithms.	Includes components, layers and their relationships.
Deals with lower-level technical details.	Deals with major structural decisions.

College Study
Your Study Hub

Software Design	Software Architecture
Helps developers implement individual modules.	Helps developers understand the complete system structure.

Example:

For an online shopping application:

- Architecture: Three-tier architecture consisting of presentation, business logic and database layers.
- Design: Detailed design of the shopping cart, payment method and product search functions.

(c) Ad hoc Testing vs Pair Testing

(Unit 4 – Must prepare)

Ad hoc Testing	Pair Testing
Informal testing without predefined test cases.	Testing performed by two people together.
Usually performed by an individual tester.	Requires two participants.
Depends on tester experience and intuition.	Uses collaboration and shared knowledge.
No fixed testing procedure is necessary.	Participants coordinate their testing activities.
Focuses on finding unexpected defects.	Focuses on improving defect detection through collaboration.
Example: Randomly testing form inputs.	Example: Two testers testing a payment module together.

(d) Exploratory Testing vs Iterative Testing

(Unit 4 – Must prepare)

Exploratory Testing	Iterative Testing
Testing and learning happen simultaneously.	Testing is performed repeatedly across development iterations.
Testers explore the application dynamically.	Testing follows a repeated development and testing cycle.
Test cases can change during execution.	Test cases are prepared for each iteration.
Focuses on discovering unexpected defects.	Focuses on continuous verification and improvement.
Highly flexible testing approach.	Organized around development iterations.
Example: Exploring a new mobile application.	Example: Testing every new feature in successive releases.

Important distinction: Exploratory testing describes how testing is conducted, whereas iterative testing describes the repeated testing process across development cycles.

(e) Defect Seeding vs Unit Testing

(Unit 4 – Must prepare)

Defect Seeding	Unit Testing
Artificial defects are deliberately introduced into software.	Individual units of software are tested.
Evaluates the effectiveness of testing.	Verifies correctness of individual components.
Focuses on measuring defect detection capability.	Focuses on detecting actual defects in a unit.
Artificial errors are intentionally created.	Real implementation errors are identified.
Used to evaluate testing quality.	Used to verify individual functions or modules.
Example: Introducing 10 known defects to measure detection rate.	Example: Testing a function that calculates student marks.

PYQ – Question 3: Practical Case Study (5 Marks)

(Important – Prepare this answer thoroughly)

Question summary

HB Corporation uses an application called HBMemo for issuing electronic memos between employees.

The application uses predefined templates and Electronic Data Interchange (EDI).

The original API is expensive to use for upgrading the system, so Indian developers use Robotic Process Automation (RPA) to automate existing manual processes without relying on those APIs.

After upgrading the system, developers ask you to perform testing.

The question asks:

1. How will you execute the testing process?
2. What general factors will you target?
3. How will you quote the testing cost?
4. What will be your testing plan of action?

Answer:

Introduction

Software testing is an important activity performed after upgrading an existing application to ensure that the modified system works correctly and that previously available functionalities are not affected.

In this case, HB Corporation has upgraded its HBMemo application using Robotic Process Automation (RPA). Therefore, I will perform a combination of functional testing, integration testing, system testing, regression testing and acceptance testing.

1. How will you execute the testing process?

I will follow these steps:

Step 1: Requirement Analysis

- Understand the existing HBMemo application.
- Study the newly automated processes.
- Identify the changes made by developers.
- Understand the expected behaviour of the upgraded system.

Step 2: Test Planning

- Define testing objectives.
- Identify testing scope.
- Select testing techniques.
- Allocate testing resources.
- Prepare a testing schedule.

Step 3: Test Case Preparation

Prepare test cases for:

- Employee login.
- Memo creation.
- Memo generation using templates.
- Sending and receiving memos.
- EDI functionality.
- RPA automation.
- Error handling.

Step 4: Functional Testing

Verify whether all existing and newly automated functionalities are working correctly.

For example, check whether the system generates and sends employee memos correctly.

Step 5: Integration Testing

Check whether RPA components communicate correctly with the existing HBMemo modules, databases and EDI functionality.

Step 6: Regression Testing

Verify that the newly introduced automation has not affected previously working functionalities.

Step 7: Performance and Reliability Testing

Check whether the upgraded application works efficiently under normal and heavy workloads.

Step 8: Acceptance Testing

Ask selected employees to use the upgraded application and verify whether it meets their requirements.

Step 9: Defect Reporting and Retesting

Record identified defects, assign priorities, fix the issues and execute the relevant test cases again.

2. What general factors will you target during testing?

I will focus on the following factors:

1. Functionality: Verify whether all memo-related functions work correctly.
2. Performance: Check the response time of the upgraded system.
3. Reliability: Ensure that the application operates consistently without unexpected failures.
4. Security: Verify that only authorized employees can access confidential memos.
5. Compatibility: Check compatibility with supported browsers, operating systems and devices.
6. Integration: Verify communication between RPA, EDI and existing system modules.
7. Accuracy: Ensure that memos contain correct information and predefined templates are properly used.
8. Usability: Check whether employees can easily use the upgraded application.
9. Error Handling: Verify how the system responds to invalid inputs, network failures and automation errors.
10. Data Integrity: Ensure that memo data is not lost, duplicated or modified incorrectly.

3. How will you quote the testing cost?

Testing cost will be estimated according to the resources, time and activities required to complete the testing process.

The following factors will be considered:

- Size and complexity of the application.
- Number of modules and test cases.
- Number of testers required.
- Testing duration.
- Cost of testing tools and automation software.
- Hardware and infrastructure requirements.
- Cost of test environment setup.
- Defect fixing and retesting effort.
- Maintenance and reporting costs.

A simple estimation formula is:

$$
\text{Total Cost} =
\text{Human Resource Cost}
+\text{Tool Cost}
+\text{Infrastructure Cost}
+\text{Other Costs}
$$

Human resource cost can be estimated as:

$$

\text{Testing Effort (Hours)}

\times

\text{Hourly Rate}

\]

A contingency amount should also be considered for unexpected testing activities.

4. What will be your plan of action concerning testing?

I will follow the following action plan:

Phase	Activity
Phase 1	Requirement analysis
Phase 2	Test planning and estimation
Phase 3	Test case design
Phase 4	Testing environment setup
Phase 5	Functional and integration testing
Phase 6	System and regression testing
Phase 7	Performance and security testing
Phase 8	Acceptance testing
Phase 9	Defect reporting and retesting
Phase 10	Final test report and approval

Conclusion:

My main objective will be to ensure that the upgraded HBMemo application works correctly, all RPA-based automated processes perform as expected, existing functionalities remain unaffected and the system is reliable, secure and ready for production use.

---

4. Most Expected Questions for This Mid-Sem

Based on your syllabus and previous-year paper, these are the questions I would prepare first.

(Priority A – Prepare completely)

No.	Expected question	Unit
1	Explain Top-Down, Bottom-Up and Bi-directional Integration Testing.	3
2	Explain System Integration Testing with an example.	3
3	Differentiate Functional and Non-functional Testing.	3
4	Explain Scenario Testing and Defect Bash.	3
5	Explain Acceptance Testing and its types.	3
6	Explain Scalability, Reliability and Stress Testing.	3
7	Explain Regression Testing and its complete process.	4
8	Differentiate Smoke Testing and Sanity Testing.	4
9	Explain Regression Test Selection techniques.	4
10	Explain Dynamic Slicing with an example.	4
11	Explain Test Minimization and its techniques.	4
12	Explain Exploratory Testing, Pair Testing and Ad hoc Testing.	4

No.	Expected question	Unit
13	Explain Deployment Testing and Beta Testing.	3
14	Explain Acceptance Criteria with an example.	3
15	Explain Test Case Selection and Execution.	3
16	Explain Execution Trace and its importance.	4
17	Explain Defect Seeding with an example.	4
18	Explain tools used for Regression Testing.	4
19	Explain Iterative Testing with an example.	4
20	Explain the difference between Static and Dynamic Slicing.	4

5. Additional differences you should not skip

These are easy-to-prepare questions and can be asked as short differences.

Comparison	Unit
Top-Down vs Bottom-Up Testing	3
Alpha vs Beta Testing	3
Integration vs System Testing	3
System Testing vs Acceptance Testing	3
Functional vs Non-functional Testing	3
Smoke vs Sanity Testing	4
Regression Testing vs Retesting	4
Ad hoc vs Exploratory Testing	4
Exploratory vs Iterative Testing	4
Defect Seeding vs Unit Testing	4
Static vs Dynamic Slicing	4
Test Selection vs Test Minimization	4

Some additional differences – Quick revision

Regression Testing vs Retesting

Regression Testing	Retesting
Checks whether existing functionalities are affected by changes.	Checks whether previously reported defects have been fixed.
Tests related existing functionalities.	Tests the specific failed test cases again.
May include many test cases.	Focuses on previously failed test cases.
Performed after modifications.	Performed after defect correction.

Alpha Testing vs Beta Testing

Alpha Testing	Beta Testing
Performed at the developer's site.	Performed by external users in real environments.
Usually conducted before beta testing.	Conducted after alpha testing.

College Study
Your Study Hub

Alpha Testing	Beta Testing
Internal testers or selected users participate.	Selected external users participate.
Developers may be available to fix defects.	Feedback is collected for further improvements.

Integration Testing vs System Testing

Integration Testing	System Testing
Tests interactions between combined modules.	Tests the complete integrated application.
Focuses on interfaces and communication.	Focuses on overall system requirements.
Performed after unit testing.	Performed after integration testing.
Example: Checking communication between payment and order modules.	Example: Testing the complete online shopping application.

6. Important diagrams to practise

Diagrams can make your 5-mark answers more presentable and help explain the testing process.



Diagram: top down bottom up integration testing diagram software engineering

1. Integration Testing

Top-down and bottom-up approaches.



Diagram: software testing levels unit integration system acceptance testing diagram

2. Testing Levels

Unit, integration, system and acceptance testing.

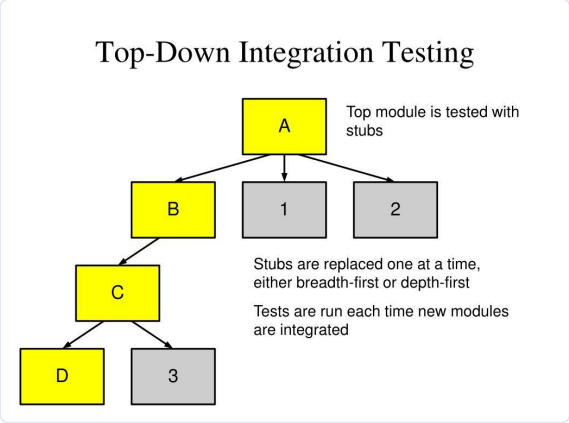


Diagram: regression testing process flowchart diagram

3. Regression Testing

Change analysis to retesting.

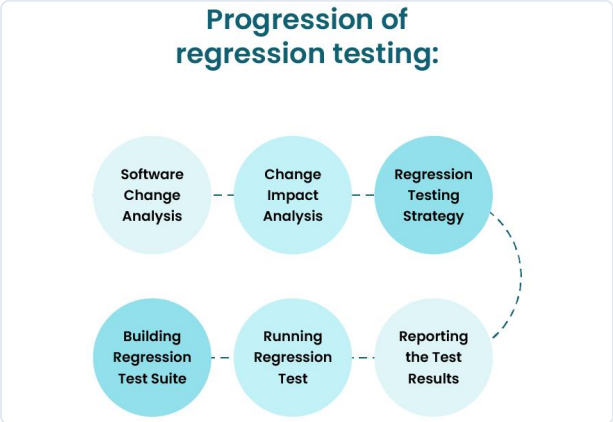


Diagram: software testing lifecycle STLC flowchart diagram

4. Testing Life Cycle

Planning, designing, execution and reporting.

7. How to prepare these two units effectively

If you have limited time, follow this order:

Preparation session	Topics
Session 1	Integration Testing and its approaches
Session 2	System, Scenario and Defect Bash
Session 3	Functional/Non-functional, Deployment and Beta Testing
Session 4	Scalability, Reliability, Stress and Acceptance Testing
Session 5	Regression Testing and its complete process
Session 6	Smoke, Sanity and Regression Test Selection
Session 7	Execution Trace, Dynamic Slicing and Test Minimization
Session 8	Ad hoc, Pair, Exploratory, Iterative and Defect Seeding

8. How to write answers in the examination

For a 5-mark question, follow this structure:

1. Definition: Start with a clear definition (2–3 lines).
2. Explanation: Explain the concept in detail.
3. Types/Steps: Use numbered points wherever applicable.
4. Example: Give one practical example.
5. Advantages: Mention 3–4 advantages if relevant.
6. Diagram: Draw a diagram wherever possible.
7. Conclusion: Finish with one concluding sentence.

For difference-based questions, always use a comparison table with at least 5 points.

---

9. Final revision checklist

Your preparation tracker

({done.length}/{tasks.length} completed)

{#each tasks as task,i}

setDone(old=>v?[...old,i]:old.filter(x=>x!==i))}>{task}

{/each}

setDone([])}>Reset checklist

My final recommendation: Do not study only definitions. Unit 3 is especially important for integration approaches, different levels of testing, acceptance testing and performance-related testing. Unit 4 is especially important for regression testing, test selection, dynamic slicing and test minimization.

Also, prepare the HBMemo case study properly. Its format is very suitable for another application-based question involving an upgraded system.

One more thing: Your previous-year paper includes some Unit 5 topics. Since you have confirmed that only Units 3 and 4 are coming, do not spend more time on those out-of-syllabus topics unless your professor specifically includes them.

You should aim to understand the examples and steps so that even if the examiner changes the wording of a question, you can frame the answer yourself.

If you want, I can:

- Explain more on Test Case Selection strategies
- Describe more examples of Defect Bash
- Expand on differences between Smoke and Sanity Testing